6章

複雑な業務ロジックに立ち向かう

前の章で、比較的単純な業務ロジックを扱う二つの方法、つまりトランザクションスクリプトとアクティブレコードを検討しました。この章も引き続き業務ロジックの実装方法を検討します。この章で紹介するのは、複雑な業務ロジックの実装方法、つまりドメインモデルです。

6.1　歴史

ドメインモデルは、マーチン・ファウラーが『エンタープライズアプリケーションアーキテクチャパターン』で紹介した、業務ロジックの実装方法の一つです。トランザクションスクリプトとアクティブレコードもこの本で紹介されています。ファウラーは、ドメインモデルの説明の最後で、「エリック・エヴァンスがドメインモデルの作り方について本を執筆中である」と書いています。この執筆中の本こそ『エリック・エヴァンスのドメイン駆動設計』です。

エヴァンスはこの本で、ソースコードとその根底にある事業活動のモデルをしっかりと関連づける手法として、集約、値オブジェクト、リポジトリなどを提示しました。これらの手法は、ファウラーが書き残した内容を忠実に踏まえています。そして、ドメインモデルを実装するための、効果的な工具セットとなっています。

エヴァンスが紹介したこれらの手法は「戦術的なドメイン駆動設計」と呼ばれることがあります。ドメイン駆動設計に取り組むことと、業務ロジックの実装にエヴァンスが紹介した手法を使うことを混同しないように、私はファウラーの用語にこだわりたいと思います。業務ロジックの実装方法は「ドメインモデル」です。集約と値オブジェクトは、ドメインモデルを実装するための部品です。

6.2　ドメインモデル

ドメインモデルは複雑な業務ロジックを扱うための設計手法です。この章では、データベースのCRUD操作ではなく、複雑な状態遷移、業務ルールそして不変条件、つまり常に保護されるべきルール、を扱います。

ヘルプデスクシステムを考えてみましょう。サポートチケットの状態を管理するロジックは、たとえば、次のようになるでしょう。

- 利用者はサポートを依頼するチケットを起票（オープン）する
- 利用者とサポート担当者は、チケットにメッセージを追加でき、すべてのやりとりを追跡できる
- チケットには四つの優先度（低、普通、高、緊急）がある
- サポート担当者は、チケットの優先度ごとに、定められた期限内に回答する必要がある（SLA：Service Level Agreement）
- 担当者がSLAの期限内に回答できなかった場合には、利用者はサポート担当者の管理者に対応を要求できる（エスカレートできる）
- エスカレートされたチケットの対応時間は三分の二に短縮される
- エスカレート後に設定された対応時間の半分を過ぎた時点で、エスカレートされたチケットを担当者がオープンしていない場合、そのチケットを自動的に他の担当者に割り当てる
- 担当者が利用者に質問してから7日間たっても利用者から返事がない場合、そのチケットは自動的にクローズされる
- エスカレートされたチケットは自動的なクローズができない。担当者もクローズできない。クローズできるのは利用者か管理者だけである
- チケットがクローズされてから7日以内であれば、利用者はチケットを再オープンできる

チケットの状態管理に関するさまざまなルールが複雑に絡み合っています。これは前の章で検討したCRUD画面の実装ではありません。この業務ロジックをアクティブレコードで実装すれば、あちこちにロジックが重複します。業務ルールの実装にまちがいがあれば、チケットの状態に不整合が発生するでしょう。

6.2.1　実装方法

ドメインモデルとは、ロジックとデータの両方を一体化させた、事業活動を表現するオブジェクトモデルです**＊1**。ドメイン駆動設計の戦術的手法、つまり集約、値オブジェクト、業務イベントそして業務サービスは、このオブジェクトモデルを組み立てるための部品です**＊2**。

＊1　マーチン・ファウラー著『エンタープライズアプリケーションアーキテクチャパターン』（翔泳社、2005年）。

＊2　この章のサンプルコードはオブジェクト指向プログラミング言語（C#）を使います。しかし、ここで検討してる考え方は、オブジェクト指向プログラミングに限定されるものではありません。関数的なプログラミングでも適用可能です。

ドメインモデルを構成するこれらの部品に共通する目的は、業務ロジックを第一に考えることです。さまざまな設計課題に対して、ドメインモデルという実装方法がどのように対処するかを見てみましょう。

6.2.1.1　複雑さ

事業活動の業務ロジックは本質的に複雑です。ですからドメインモデルにそれ以上の不必要な複雑さを持ち込むべきではありません。データベース操作や外部との通信など、技術的な関心事からドメインモデルを完全に切り離すことが必要です。そうするためにドメインモデルは「古くからある単純なオブジェクト」（Plain Old Object）だけで組み立てます。実行基盤に関係するライブラリやフレームワークに依存させてはいけません＊3。

＊3　.NETだとPOCO、JavaだとPOJO、PythonだとPOPOなどです。

6.2.1.2　同じ言葉

技術的な関心事ではなく、業務ロジックに焦点を合わせることで、ドメインモデルを構成するオブジェクトは区切られた文脈の同じ言葉の用語と対応しやすくなります。別の言い方をすれば、ドメインモデルでは、ソースコードが同じ言葉を語り、業務エキスパートのメンタルモデルを表現します。

6.2.2　ドメインモデルの部品

ドメインモデルを組み立てる時に中心となる部品、つまり値オブジェクト、集約、業務サービスを見てみましょう。

6.2.2.1　値オブジェクト

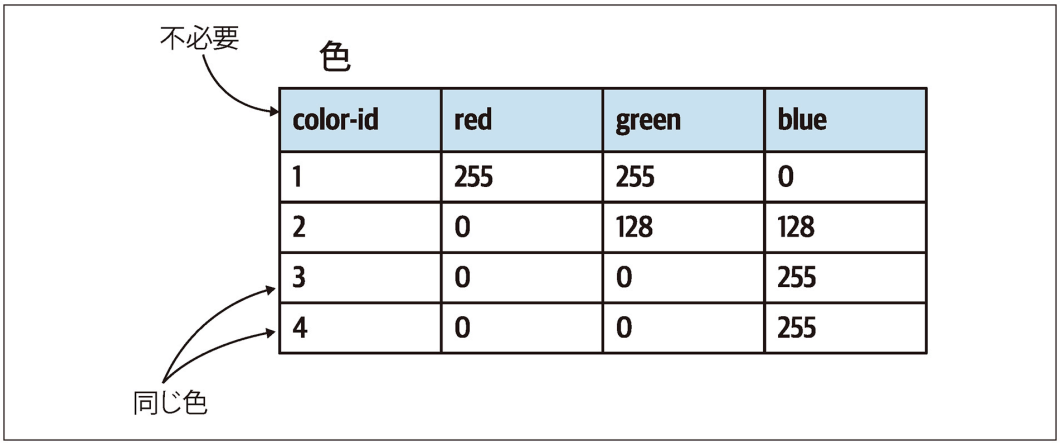
値オブジェクト（Value Objects）は、オブジェクト内部のフィールド値の組み合わせによって識別されるオブジェクトです。例として、色を表現するオブ

ジェクトを考えてみましょう。

```
class Color
{
  int _red;
  int _green;
  int _blue;
}
```

三つのフィールド（`_red`、`_green`、`_blue`）の値の組み合わせで、色を定義します。いずれかのフィールドの値を変えれば、別の色になります。異なる色が、三つのフィールド値の同じ組み合わせになることはありません。そして、同じ色を表現する二つのオブジェクトどうしでは、三つのフィールドの値は完全に一致します。したがって、色の違いを識別するための、明示的な識別フィールドは必要はありません。

図6‐1の`color-id`は必要ありません。不具合の原因になるだけです。この表の形式だと、`red`、`green`、`blue`の三つ値がすべて同じになる行、つまり同じ色の行をいくつも作れます。しかし、3行目と4行目の`color-id`の値は、二つの行が同じ色であることを正しく表現していません。



color-id	red	green	blue
1	255	255	0
2	0	128	128
3	0	0	255
4	0	0	255

図6-1　不必要な`color-id`があるために同じ色を重複して定義できてしまう

同じ言葉

プログラミング言語の標準ライブラリに用意されたデータ型（文字列型、整数型、ディクショナリ型）だけを使って事業活動の概念を表現するやり方は、「基本データ型への執着」と呼ばれるコードのいやな臭いです＊4。たとえば次のクラスです。

＊4　引用元：https://wiki.c2.com/?PrimitiveObsession（2021年6月13日参照）

```
class Person
{
  private int  _id;
  private string _firstName;
  private string _lastName;
  private string _landlinePhone;
  private string _mobilePhone;
  private string _email;
  private int  _heightMetric;
  private string _countryCode;

  public Person(...) {...}
}

static void Main(string[] args)
{
  var dave = new Person(
    id: 30217,
    firstName: "Dave",
    lastName: "Ancelovici",
    landlinePhone: "023745001",
    mobilePhone: "0873712503",
    email: "dave@learning-ddd.com",
    heightMetric: 180,
    countryCode: "BG");
}
```

このPersonクラスはフィールドのほとんどを文字列型（string）で実装しています。そして慣例となっている表現形式で値を設定します。たとえばlandlinePhoneの値は有効な固定電話番号の形式であることが必要で、countryCodeはアルファベット大文字二桁で、有効な国コードでなければいけません。もちろん、まちがった形式の文字列が入力される可能性があります。そのため、このクラスはすべての入力値の妥当性を検査する必要があります。

このクラス設計はさまざまな問題があります。まず、妥当性を検査するロジックがあちこちに重複するでしょう。また、値が必ず検査済であることを保証

するのは難しいでしょう。そして、後から別の技術者がコードを変更することでさらに状況は悪化するでしょう。

では値オブジェクトを使って実装すると、どうなるでしょうか。次の設計を前の設計と比較してみましょう。

```
class Person {
  private PersonId     _id;
  private Name         _name;
  private PhoneNumber  _landline;
  private PhoneNumber  _mobile;
  private EmailAddress _email;
  private Height       _height;
  private CountryCode  _country;

  public Person(...) { ... }
}

static void Main(string[] args)
{
  var dave = new Person(
    id:       new PersonId(30217),
    name:     new Name("Dave", "Ancelovici"),
    landline: PhoneNumber.Parse("023745001"),
    mobile:   PhoneNumber.Parse("0873712503"),
    email:    Email.Parse("dave@learning-ddd.com"),
    height:   Height.FromMetric(180),
    country:  CountryCode.Parse("BG"));
}
```

第一に、意図が明確になりました。たとえば`country`変数です。国を表現する値として、完全な国名ではなく国コードを使う、という設計意図を伝えるために、`countryCode`という手の込んだ変数名を使う必要はありません。変数名が単純でも、値オブジェクトのクラス名が意図を明確に表現します。

第二に、`country`変数に値を設定するたびに、値の妥当性を検証する必要はありません。なぜならば、検証ロジックは値オブジェクト自身が持ちます。値オブジェクトの役割は値の妥当性検査にとどまりません。値オブジェクトが真価を発揮するのは、その値を使う業務ロジックを一元的に記述した時です。強く関係するロジックが1箇所で実装され、テストもやりやすくなります。もっとも重要なことは、値オブジェクトが業務の概念を表現することです。つまり、値オブジェクトを使うことで、ソースコードが同じ言葉を話します。

身長、電話番号、色などの概念を値オブジェクトで表現することで、型の体系がどのくらい豊かになり直感的に使えるようになるか見てみましょう。

身長を表現する値オブジェクト`Height`を使うほうが、整数型を使うよりも意図が明確です。また身長の表現と測定単位の関係を切り離せます。たとえば、以下のコード例のように、値オブジェクト`Height`は、メートル法（Metric）とヤード法（Imperial）それぞれのオブジェクトを生成し、測定単位の変換、単位ごとの異なる文字列表現、そして異なる単位どうしの身長の比較を簡単に実装できます。

```
var heightMetric = Height.Metric(180);              // メートル法で初期化
var heightImperial = Height.Imperial(5, 3);         // ヤード法で初期化

var string1 = heightMetric.ToString();              // "180cm"
var string2 = heightImperial.ToString();            // "5 feet 3 inches"
var string3 = heightMetric.ToImperial().ToString(); // "5 feet 11 inches'

var firstIsHigher = heightMetric > heightImperial;  // true:異なる単位で比
```

電話番号を表現する値オブジェクト`PhoneNumber`は、電話番号の文字列表現からの読み取り、妥当性の検証、そしてさまざまな特性（国や電話種別）の判定などのロジックをカプセル化できます。たとえば以下のようなロジックです。

```
var phone = PhoneNumber.Parse("+359877123503");
var country = phone.Country;                        // "BG"
var phoneType = phone.PhoneType;                    // "MOBILE"
var isValid = PhoneNumber.IsValid("+972120266680"); // false
```

次の例は、データ操作や適切なインスタンスの生成という、すべての業務ロジックをカプセル化した時の、値オブジェクトのパワーを示しています。

```
var red = Color.FromRGB(255, 0, 0);   // 赤
var green = Color.Green;              // 緑
var yellow = red.MixWith(green);      // 黄（混合結果）
var yellowString = yellow.ToString(); // "#FFFF00"
```

ここまで見てきたように、値オブジェクトを使えば暗黙的な決めごとの知識は不要になります。たとえば、ある文字列が電子メールアドレスなのか電話番

号なのかを覚えておく必要はありません。値オブジェクトを使えば、オブジェクトモデルの意図が理解しやすくなり、直感的に使えるようになります。

実装方法

値オブジェクトはフィールドを変更すると、元の値が別の値に変わってしまいます。そうならないように、値オブジェクトは不変（イミュータブル）なオブジェクトとして実装します。値オブジェクトのフィールドを変更したい時は、別の値、つまり別のインスタンスを生成します。以下のコード例の `MixWith` メソッドのように、値に対するアクションの結果が別の値になる場合、元のインスタンスはそのままにして、新たなインスタンスを生成して返します。

```
public class Color
{
  public readonly byte Red;
  public readonly byte Green;
  public readonly byte Blue;

  public Color(byte r, byte g, byte b)
  {
    this.Red = r;
    this.Green = g;
    this.Blue = b;
  }

  public Color MixWith(Color other)
  {
    return new Color( //別のインスタンスを生成
      r: (byte) Math.Min(this.Red + other.Red, 255),
      g: (byte) Math.Min(this.Green + other.Green, 255),
      b: (byte) Math.Min(this.Blue + other.Blue, 255)
    );
  }

  ...
}
```

値オブジェクトが等しいかどうかは、 `id` フィールドや参照ポインタが同じかどうかではなく、値が同じかどうかで判定します。ですから、値オブジェクトを正しく実装するには、オブジェクトの同一性の判定メソッドのオーバーライドが重要です。たとえば、C#では次のようにします[*5]。

＊5　C# 9.0で新たに導入された`record`型は値にもとづく同一性を判定します。したがって、同一性の判定メソッドや演算子をオーバーライドする必要はありません。

```
public class Color
{
  ...

  public override bool Equals(object obj)
  {
    var other = obj as Color;
    return other != null &&
      this.Red == other.Red &&
      this.Green == other.Green &&
      this.Blue == other.Blue;
  }

  public static bool operator == (Color lhs, Color rhs)
  {
    if (Object.ReferenceEquals(lhs, null)) {
      return Object.ReferenceEquals(rhs, null);
    }
    return lhs.Equals(rhs);
  }

  public static bool operator != (Color lhs, Color rhs)
  {
    return !(lhs == rhs);
  }

  public override int GetHashCode()
  {
    return ToString().GetHashCode();
  }

  ...
}
```

標準ライブラリの`String`を使って対象領域に固有の値を表現することは、値オブジェクトの考え方に反してはいますが、.NETやJavaなどのプログラミング言語に用意されている`String`型は、まさに値オブジェクトとして実装されています。Stringは不変（イミュータブル）で、すべての操作は新しいインス

タンスを生成します。さらに言えば、String型は一つまたは複数の文字列を使って新たな文字列を生成する、さまざまな操作をカプセル化します。たとえば、trim、concatenate、replace、substringなどの操作です。

値オブジェクトをいつ使うか

簡単な答えは「使える時はいつでも使う」です。値オブジェクトを使うことでコードの意図が明確になります。また業務ロジックの記述があちこちに散らばるのを防ぎます。そして値オブジェクトはコードを安全にします。不変（イミュータブル）な値オブジェクトのメソッドには副作用がありません。複数スレッドを同時並行で実行しても問題は起きません。

事業活動の観点からは、値オブジェクトは事業活動を表現する基本部品と考えるとよいでしょう。具体的には次節で検討するエンティティの属性として値オブジェクトを使います。先ほどの例では`Person`を表現する属性（ID、名前、電話番号、メールアドレスなど）に値オブジェクトを使いました。値オブジェクトを使うその他の例としては、状態区分やパスワードなど、対象領域に固有の概念があります。これらを値オブジェクトで表現した場合、同一性の判定は値の比較によって行います。明示的なIDフィールドを必要としません。値オブジェクトの用途として特に重要なのが、通貨あるいは金銭的な価値の表現です。金額を`int`などプリミティブな型で扱うと、金額に関わる業務ロジックがあちこちに散らばり、関連する業務ロジックを1箇所にまとめるカプセル化が難しくなります。そして丸めや端数処理に危険な不具合が紛れ込みやすくなります。

6.2.2.2　エンティティ

エンティティ（Entity）は値オブジェクトとは対照的です。エンティティは個々のインスタンスを特定するための識別情報（ID）が必要です。簡単な例として人（person）をエンティティとして表現してみましょう。以下のコードを考えてみましょう。

```
class Person
{
  public Name Name { get; set; }

  public Person(Name name)
  {
    this.Name = name;
```

```
    }
  }
```

このクラスのフィールドは一つだけ、つまり、値オブジェクトを使って表現した`name`だけです。しかしこの設計は十分ではありません。同じ名前の人は何人もいるでしょう。名前が同じだからといって同一人物とは限りません。したがって、以下のコードのように、個人を一意に特定できる識別情報が必要です。

```
class Person
{
  public readonly PersonId Id;
  public Name Name { get; set; }

  public Person(PersonId id, Name name)
  {
    this.Id = id;
    this.Name = name;
  }
}
```

`PersonId`型の識別フィールドを追加しました。`PersonId`は値オブジェクトです。その内部では業務に応じた適切なデータ型を使えます。たとえば、`Id`はGUIDでもよいし、整数や文字列でもよいでしょう。あるいは保険証番号など対象領域に固有の値を使うことも可能です。

個体識別フィールドの値は、図6‐2のように、エンティティのインスタンスごとに一意でなければいけません。さらに言えば、よほど特殊な場合を除き、エンティティを識別する値はその個体が存在する限り変更してはいけません。この点がエンティティと値オブジェクトのもう一つの違いにつながります。

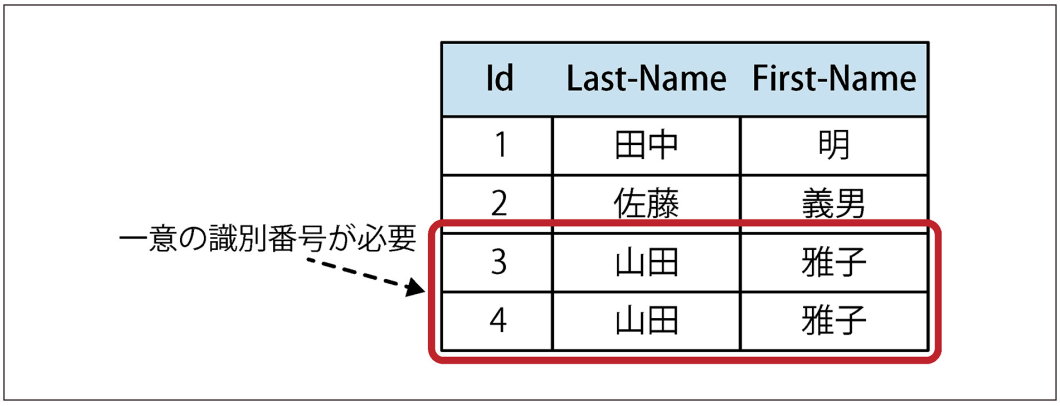


図6-2　他のフィールドの値が同じでも識別できるように一意の番号を導入する

値オブジェクトとは異なり、エンティティは不変（イミュータブル）ではありません。そしてエンティティと値オブジェクトのもう一つの違いは、値オブジェクトはエンティティの状態を表現する手段だということです。さきほど説明したように、エンティティ`Person`は二つの値オブジェクト、つまり`PersonId`と`Name`を持ちます。

エンティティは、どんな事業領域でもドメインモデルの基本となる部品です。ですが、**第1章**ではドメインモデルを組み立てるための部品リストにエンティティを含めていなかったことにお気づきでしょうか。これは意図的です。部品リストに含めなかった理由は、エンティティを単独で実装することがないためです。エンティティは必ず集約の実装の一部になります。

6.2.2.3　集約

集約（Aggregates）はエンティティです。つまり、集約は一意に識別できるフィールドが必要です。そして集約のインスタンスはそのライフサイクルの途中で状態が変化します。しかし、集約は単なるエンティティではありません。集約の目標は、データの一貫性の保証です。集約の状態は変更可能なので、常に一貫した状態を保証することは、さまざまな意味合いと課題があります。

一貫性を強制する

集約の状態は変更可能です。そのため、さまざまな状況で、データの一貫性が失われる可能性があります。データの一貫性を強制するために、集約は、内部と外部の間に明確な境界を定義します。つまり、集約とはデータの一貫性を強制する境界です。集約に記述するロジックは、状態を変更しようとする外部からのすべての操作の妥当性を検証し、業務ルールに違反した状態になることを防ぐ必要があります。

実装の観点では、集約内部の業務ロジックだけが状態を変更できるようにすることで、データの一貫性を強制します。外部のプロセスやオブジェクトには集約の状態を参照することだけを許可します。集約の状態を変更するには、集約が公開している適切なメソッドを実行することが必要です。

集約が公開インターフェースとして外部に提供する状態変更メソッドを「コマンド」（命令）と呼びます。コマンドの実装方法は二つあります。一つは通常の公開メソッドとして実装する、次のような方法です。

```
public class Ticket
{
  ...

  public void AddMessage(UserId from, string body)
  {
    var message = new Message(from, body);
    _messages.Append(message);
  }

  ...
}
```

もう一つのやり方は、コマンドをパラメーターオブジェクト（parameter object）（**https://oreil.ly/4hNtn**）として表現するやり方です。パラメーターオブジェクトとは、コマンドの実行に必要な情報をカプセル化したオブジェクトです。パラメーターオブジェクトを使ったやり方は、次のようになります。

```
public class Ticket
{
  ...

  public void Execute(AddMessage cmd)
  {
    var message = new Message(cmd.from, cmd.body);
    _messages.Append(message);
  }

  ...
}
```

集約を実装する時に、どちらの方法でコマンドを表現するかは考え方しだいです。個人的には、コマンドの種類ごとにパラメーターオブジェクトの型を定義し、多重的定義した`Execute`メソッドに適切な型のパラメーターオブジェクトを渡す、多態（ポリモーフィック）を使う実装のほうが、意図が明確になり好ましいと考えています。

集約の公開インターフェースは、外部から渡された内容を検証し、守るべき業務ルールと不変条件に違反しないようにする責任があります。また、この厳密な境界は、その集約に関連するすべての業務ロジックを1箇所、つまり、その集約自身に実装することを保証します。

集約に業務ロジックを集めると、集約を使う側のアプリケーション層＊6のクラスの実装はかなり単純になります＊7。つまり、アプリケーション層の役割は、次のような操作だけになります。

＊6　サービス層とも呼ばれます。公開API経由の操作要求をドメインモデルに転送する役割のクラス群です。

＊7　基本的にはアプリケーション層のクラスはトランザクションスクリプトです。このクラスのメソッドは複数の操作を一つのトランザクションとして管理しなければいけません。集約オブジェクトの状態の変更は、完全に成功するか、まったく変更されないかのどちらかです。中途半端な状態がデータベースに書き込まれません。

- 最新の状態を反映した集約オブジェクトを生成する
- 生成した集約に対して必要な操作を実行する
- 変更された状態を永続化する
- 操作の結果を呼び出し元に戻す

コード例は以下のとおりです。

```
01 public ExecutionResult Escalate(TicketId id, EscalationReason reason)
02 {
03   try
04   {
05     var ticket = _ticketRepository.Load(id);
06     var cmd = new Escalate(reason);
07     ticket.Execute(cmd);
08     _ticketRepository.Save(ticket);
09     return ExecutionResult.Success();
10   }
11   catch (ConcurrencyException ex)
12   {
13     return ExecutionResult.Error(ex);
14   }
15 }
```

11行目の並行処理の例外を検査するロジックに注目してください。集約の状態の一貫性を保証するもっとも重要な部分です＊8。複数のプロセスが同じ集約を変更する場合、最初のプロセスが更新した内容を後続のプロセスが上書きできないようにする必要があります。この場合、後から更新しようとしている

プロセスは、変更しようとしている元の状態がすでに変わっていることを検知し、最新の状態を取得し直して再実行する必要があります。

＊8　アプリケーション層はトランザクションスクリプトの集まりです。第5章で検討したように、競合する変更処理がデータの一貫性を壊さないように、並行処理を適切に管理することが必要です。

したがって、集約の状態を記録するデータベースは並行処理を管理する必要があります。単純なやり方としては、以下のコード例のように、集約にバージョン番号を持たせ、更新のたびにバージョン番号を増やしていく方法があります。

```
class Ticket
{
  TicketId _id;
  int      _version;

  ...
}
```

変更された集約をデータベースに書き込む時は、データベース上のバージョン番号と、集約を生成した時にデータベースから読み込んだバージョン番号が一致することを検査します。たとえばの次のようなSQL文になります。

```
01 UPDATE tickets
02 SET ticket_status = @new_status,
03 agg_version = agg_version + 1
04 WHERE ticket_id=@id and agg_version=@expected_version;
```

このSQL文の2行目で、チケット集約の状態を上書きしています。そして3行目でバージョン番号を一つ増加して上書きします。ただし4行目のWHERE句によって、現在のデータベース上のバージョン番号が、更新前に読み取ったバージョン番号と同じであることを検査しています。

もちろん、こういう並行処理の排他制御はリレーショナルデータベース以外の仕組みを使っても実現できます。もっと言えば、さまざまなドキュメント指向データベースが集約を扱う仕組みとして使えるようになってきています。い

ずれにせよ、集約の状態をデータベースで永続化する場合、並行処理の管理はきわめて重要です。

トランザクションの境界

集約の状態を変更できるのは集約内部に記述した業務ロジックだけです。そのため集約はトランザクションの境界として機能します。集約の状態を変える一連の処理は単一のトランザクションとしてコミットする必要があります。集約の状態が変わった場合、すべての変更を一体としてコミットするか、何も変更されないか、どちらかになります。

さらに言えば、複数の集約にまたがるトランザクションを実行してはいけません。複数の集約の状態を変更する時は、集約ごとに独立したデータベーストランザクションを実行し、別々にコミットします。

一つの集約インスタンスが一つのトランザクション単位、という原則に従うためには、集約の境界を注意深く設計し、業務ルールと不変条件を正しく取り扱うことが必要です。複数の集約への変更を単一のトランザクションとしてコミットしたいとしたら、トランザクションの境界、つまり集約の境界がまちがっているということです。

この原則はモデルの作り方に制限を課すように見えます。複数オブジェクトの状態変更を、一つのトランザクションとして処理したい場合はどうすればよいでしょうか。集約の設計で、この問題にどのように対処するかを見てみましょう。

エンティティの階層構造

この章の前半で検討したように、エンティティは独立した部品ではありません。エンティティは集約の一部として存在します。エンティティと集約の根本的な違いを見てみましょう。またエンティティがドメインモデルの部品ではなく集約の部品である理由も見てみましょう。

複数のオブジェクトを一つのトランザクションとして扱うことが必要な業務シナリオがあります。たとえば、二つのオブジェクトを同時に更新する場合、あるいは、あるオブジェクトの業務ルールが別のオブジェクトの状態に依存している場合です。

ドメイン駆動設計は「事業活動がソフトウェア設計を駆動する」という考え方です。集約も例外ではありません。複数のオブジェクトにまたがる変更を一

つのトランザクションとして扱うために、集約は図6‐3のようなエンティティの階層構造を一つのトランザクションとして扱います。

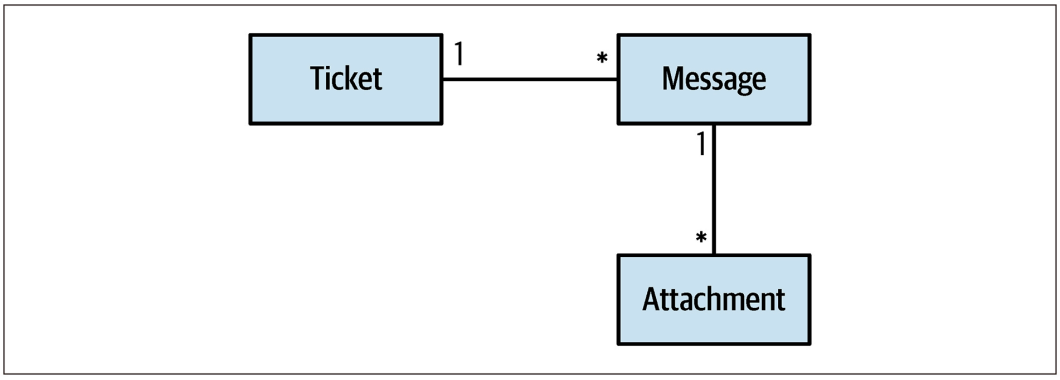


図6-3　エンティティの階層構造としての集約

この階層構造には、いくつかのエンティティと値オブジェクトが含まれます。そして、業務ロジックがすべてのオブジェクトを必要とするなら、この構造全体が一つの集約になります。

それが「集約」という名前の由来です。つまり、関連するエンティティと値オブジェクトをトランザクション境界の内部に集約します。

複数のエンティティが集まった集約を使って業務ルールを表現する例を見てみましょう。業務ルールは次のとおりです。

エスカレートされたチケットを、指定された期限の半分までに担当者が対応しなかった場合、そのチケットを他の担当者に自動的に割り当てる。

コードは次のようになります。

```
public class Ticket
{
  ...
  List<Message> _messages;
  ...

  public void Execute(EvaluateAutomaticActions cmd)
  {
    if (this.IsEscalated && this.RemainingTimePercentage < 0.5 &&
      GetUnreadMessagesCount(for: AssignedAgent) > 0)
    {
      _agent = AssignNewAgent();
    }
```

```
14   }
15
16   public int GetUnreadMessagesCount(UserId id)
17   {
18     return _messages.Where(x => x.To == id && !x.WasRead).Count();
19   }
20
21   ...
22 }
```

このメソッドの9行目で「チケットがエスカレートされたものか」と「指定された対応期限の50%が経過したか」を判定しています。さらに10行目で「現在の担当者が読んでいないメッセージがあるか」を確認します。すべての条件が成立した場合、このチケットを別の担当者に割り当てます。

この集約は、判定に使うデータ（`IsEscalated`、`RemainingTimePercentage`、`_messages`）の強い一貫性によって、条件が正しく判定されることを保証します。集約のデータへのすべての変更を単一のトランザクションとして実行することで、一連の判定処理が完了した後で変更されることはありません。

他の集約を参照する

集約に含まれるすべてのオブジェクトが一つのトランザクション単位になります。そのため集約が大きくなるとパフォーマンスやスケーラビリティの問題が起きやすくなります。

集約の境界を設計する指針として、データの一貫性が役に立ちます。集約の業務ロジックが強い一貫性を必要とする情報だけが、集約に含まれるべきです。結果的に整合していればよい情報は、集約の境界の外側に置くべきです。たとえば**図6‐4**のように別の集約の一部となる情報です。

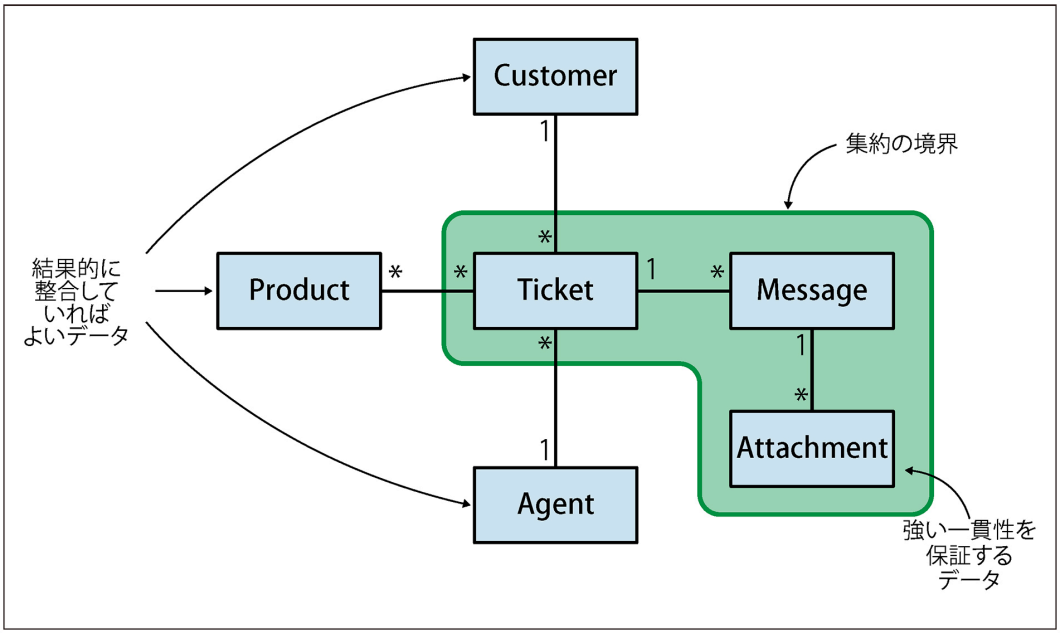


図6-4　集約によって一貫性を保証する境界

経験則として、集約はできるだけ小さく設計すべきです。以下のコード例のように、一貫性を保証する対象は、集約に記述する業務ロジックが必要とするデータだけに限定します。

```
public class Ticket
{
  private UserId          _customer;      // 顧客
  private List<ProductId> _products;      // 商品のリスト
  private UserId          _assignedAgent; // 担当者
  private List<Message>   _messages;      // メッセージのリスト
  ...
}
```

この例では、`Ticket`集約は、集約の境界の内部に保持するメッセージのリストを参照しています。一方、チケットに関係する顧客、商品のリスト、そして担当者のオブジェクトそのものは、集約の内部に保持していません。識別番号（ID）を使って参照します。

識別番号を使って外部の集約を参照する理由は、そのオブジェクトは集約の境界の外側であることを明確に表現するためです。そして、集約ごとに固有のトランザクション境界を持つことを保証します。

あるエンティティを集約で保持するかどうかを判断するには、集約に実装する業務ロジックを検討します。つまり、業務ロジックが結果整合性のみが保証

される集約外部のエンティティを参照した場合、不正な状態になる危険があるかないかで判断します。さきほどのチケットの業務ルールで考えてみましょう。対応期限の半分が経過してもメッセージを読んでいなければ、別の担当者に割り当てる、というルールです。もしメッセージの既読未読の状態が結果整合性だった場合、何が起きるでしょうか。別の言い方をすると、メッセージを読んだことを即時に検知できないとどうなるでしょうか。この場合、本来は担当者の再割り当てをすべきではない、かなりの数のチケットが誤って別の担当者に割り当てられてしまうでしょう。もちろん、これはチケット管理システムとして誤った状態です。ですから、こういう事態を防ぐために、メッセージが既読か未読かの状態は`Ticket`集約の内部で管理します。

集約のルート

集約の状態を変更できるのはコマンドを実行した時だけである、という説明をしました。集約は複数のエンティティの階層構造です。しかし外部に公開するインターフェース役のエンティティは一つにすべきです。それが集約のルートとなるエンティティです（図6‐5）。

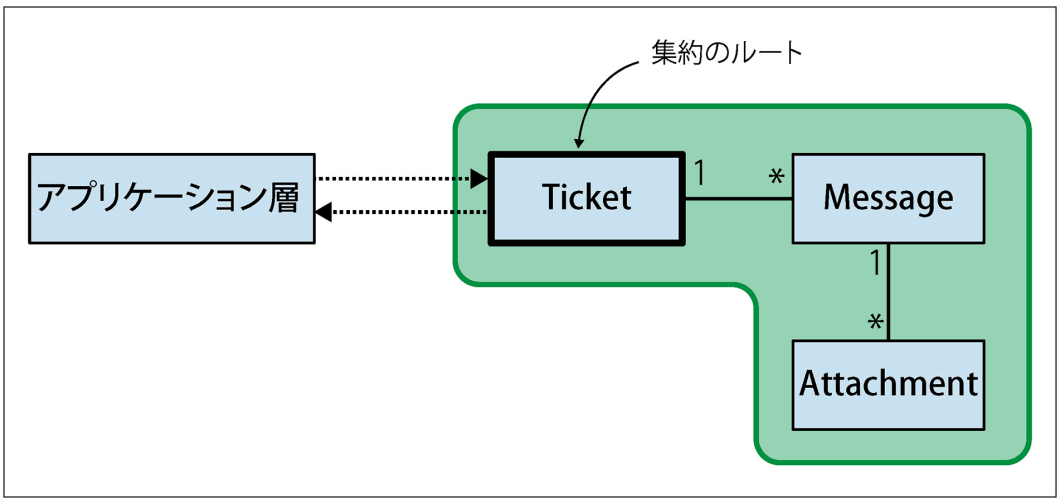


図6-5　集約のルート

`Ticket`集約の以下のコードを考えてみましょう。

```
public class Ticket
{
  ...
  List<Message> _messages;
  ...

  public void Execute(AcknowledgeMessage cmd)
  {
```

```
    var message = _messages.Where(x => x.Id == cmd.id).First();
    message.WasRead = true;
  }
  ...
}
```

この例では、`Ticket`集約は指定されたメッセージを既読に変更するコマンドを外部に公開しています。このコマンド操作は`Message`エンティティのインスタンスの状態を変更しますが、アクセスできるは、集約のルートだけです。つまり`Ticket`からしかアクセスできません。

集約のルートとなるエンティティを公開する方法とは違うやり方で、外部世界と集約を連係させる方法があります。それが業務イベントです。

業務イベント

業務イベント（Domain Events）は、事業活動の中で起きた重要な出来事を表現するメッセージです。たとえば以下のような出来事です。

- チケットを割り当てた
- チケットをエスカレートした
- メッセージを受け取った

業務イベントは実際に起きた出来事を表現します。ですから業務イベントの名前は必ず過去形になります。

業務イベントの目的は事業活動で何が起きたかを表現することです。そのイベントに関する、必要なデータすべてを提供します。たとえば、以下の業務イベントは、特定のチケットが責任者にエスカレートされたことを、発生時刻と発生理由を含めて表現しています。

```
{
  "ticket-id": "c9d286ff-3bca-4f57-94d4-4d4e490867d1",
  "event-id": 146,
  "event-type": "ticket-escalated",  // エスカレートされた
  "escalation-reason": "missed-sla", // SLA違反
  "escalation-time": 1628970815
}
```

ソフトウェア設計では「名前」が重要です。業務イベントの名前は事業活動で何が起きたかを明確に表現してください。

業務イベントは集約の公開インターフェースの一部です。集約は業務イベントを発行（publish）します。そして他のプロセス、集約、外部サービスがそのイベントを購読（subscribe）し、業務イベントに応じた業務ロジックを実行します（図6‐6）。

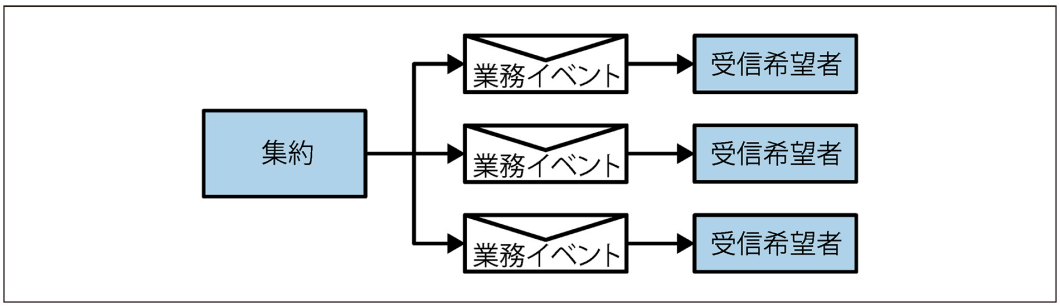


図6-6　業務イベント発行の流れ

`Ticket`集約のコードは次のようになります。業務イベントを表現するオブジェクトを生成し（12行目）、そのオブジェクトを業務イベントリスト（`_domainEvents`）に追加します（13行目）。

```
01 public class Ticket
02 {
03   ...
04   private List<DomainEvent> _domainEvents; // 業務イベントリスト
05   ...
06
07   public void Execute(RequestEscalation cmd)
08   {
09     if (!this.IsEscalated && this.RemainingTimePercentage <= 0)
10     {
11       this.IsEscalated = true;
12       var escalatedEvent = new TicketEscalated(_id, cmd.Reason);
13       _domainEvents.Append(escalatedEvent);
14     }
15   }
16
17   ...
18 }
```

第9章で、業務イベントを購読者（subscriber）に信頼性が高いやり方で発行する方法を検討します。

同じ言葉

最後になりましたが重要な点を説明します。集約を実装する時は、同じ言葉を厳密に反映することが必要です。集約の名前、フィールドに持つデータ名、メソッド名、発信する業務イベントの名前、そういうすべての名前を、区切られた文脈の同じ言葉によって命名します。エリック・エヴァンスが書いているように、ソースコードは、開発者どうしの会話や業務エキスパートとの会話で使う、同じ言葉にもとづいて書かなければいけません。このことは、複雑な業務ロジックを実装する場合、特に重要です。

次にドメインモデルを組み立てる最後の部品として業務サービスを見てみましょう。

6.2.2.4　業務サービス

集約や値オブジェクトでは表現しにくい業務ロジックが見つかることがあります。あるいは複数の集約にまたがる業務ロジックが見つかるかもしれません。ドメイン駆動設計の業務サービス（Services）は、このような業務ロジックを記述する方法です。

業務サービスは、業務ロジックだけを記述する、自分自身の状態は持たないステートレスなオブジェクトです。ほとんどの場合、業務サービスは、さまざまなコンポーネントの呼び出しを統合して、何らかの計算や分析を行います。

チケット集約の例で考えてみましょう。チケットを割り当てられた担当者は、一定の時間内に顧客に対して何らかの解決策を提案する必要があります。対応するまでに許容される時間枠は、チケットの優先順やエスカレーションの状況によって変動します。また所属部門の運営方針として決めている優先度別の対応時間にも依存します。そして、勤務していない担当者にチケットを割り当てなようにするために、サポート担当者の勤務予定も関係します。

対応までの許容時間の計算は、さまざまな情報源からのデータが必要です。つまり、チケット、担当者の所属部門、そして担当者の勤務予定です。このような計算ロジックを実装する方法として、次のような業務サービスが有力な選択肢です。

```
public class ResponseTimeFrameCalculationService
{
  ...

  public ResponseTimeframe CalculateAgentResponseDeadline(UserId agentId,
    Priority priority, bool escalated, DateTime startTime)
  {
    var policy = _departmentRepository.GetDepartmentPolicy(agentId);
    var maxProcTime = policy.GetMaxResponseTimeFor(priority);

    if (escalated) {
      maxProcTime = maxProcTime * policy.EscalationFactor;
    }

    var shifts = _departmentRepository.GetUpcomingShifts(agentId,
      startTime, startTime.Add(policy.MaxAgentResponseTime));

    return CalculateTargetTime(maxProcTime, shifts);
  }

  ...
}
```

業務サービスを使うと、複数の集約にまたがる業務ロジックをわかりやすく記述できます。しかし集約の変更とデータベーストランザクションの関係を常に意識してください。集約のインスタンスの変更は、単一のデータベーストランザクションであることが必要です。業務サービスはこの原則の抜け道ではありません。一つのインスタンスごとに一つのトランザクション、という原則は変わりません。業務サービスが役に立つのは、複数の集約を参照する計算ロジックを記述する場合です。

もう一つ重要な点があります。業務サービスはマイクロサービスやサービス指向アーキテクチャなどで使われている「サービス」という用語とは関係ありません。ドメイン駆動設計の業務サービスは、業務ロジックの置き場所として使う、状態を持たないオブジェクトのことです。

6.2.3　複雑さの扱い方

この章の最初で説明したように、集約と値オブジェクトは、業務ロジックを実装する時の、複雑さに立ち向かうための手段として導入されました。この背景にある考え方を見てみましょう。

事業経営の卓越した理論を提示したエリヤフ・ゴールドラットは、書籍『ザ・チョイス：複雑さにまどわされるな』（ダイヤモンド社、2008年）の中で、システムの複雑さについて簡潔かつ説得力のある説明をしています。ゴールドラットによれば、システムの複雑さを議論する時は、システムの振る舞いを制御し予測する難易度で判断します。この難易度は対象とするシステムの「自由度」によって決まります。

システムの自由度とは、状態を表現するデータの個数です。次の二つのクラスを考えてみましょう。

```
public class ClassA
{
  public int A { get; set; }
  public int B { get; set; }
  public int C { get; set; }
  public int D { get; set; }
  public int E { get; set; }
}

public class ClassB
{
  private int _a, _d;

  public int A
  {
    get => _a;
    set {
      _a = value;
      B = value / 2;
      C = value / 3;
    }
  }

  public int B { get; private set; }

  public int C { get; private set; }

  public int D
  {
    get => _d;
    set {
      _d = value;
      E = value * 2
    }
```

```
    }

    public int E { get; private set; }
  }
```

第一印象では、`ClassA`と比べ、`ClassB`のほうがかなり複雑に見えます。インスタンス変数の数はどちらも同じですが`ClassB`は計算ロジックを実装しています。では`ClassB`は、本当に`ClassA`より複雑でしょうか。

二つのクラスを自由度の観点から分析してみましょう。`ClassA`で状態を表現するデータ項目の数はいくつでしょうか。答えは五つです。したがって`ClassA`の自由度は5です。

`ClassB`の状態を表現するデータ項目の数はどうでしょうか。属性の`A`と`D`のロジックを調べると、属性`B`、`C`、`E`の値は、`A`と`D`の値を使った関数です。`A`と`D`の二つの値が決まれば、残りの変数の値は自動的に決まります。したがって、`ClassB`の自由度は2です。状態の表現に必要な変数は二つだけです。

最初の質問に戻りましょう。システムの振る舞いを制御し予測するのが難しいのはどちらのクラスでしょうか。答えは自由度が5の`ClassA`です。`ClassB`は、不変条件を追加することで。複雑さが小さくなっています。これが集約と値オブジェクトの背景にある考え方です。つまり、不変条件をカプセル化することで複雑さを小さくします。

値オブジェクトは、内部の状態に関係するすべての業務ロジックを、その境界の内部に記述します。集約も同じです。集約の内部の状態を変えられるのは、集約自身のメソッドだけです。業務ロジックをカプセル化し業務ルールを確実に適用することで自由度が下がります。

業務ロジックをドメインモデルで実装するのは、業務ロジックが複雑な業務領域だけです。ですから、ドメインモデルを選択するのは、ソフトウェアの心臓部である中核の業務領域だと考えておけばよいでしょう。

6.3　まとめ

ドメインモデルは複雑な業務ロジックを実装する手段です。ドメインモデルの主要部品は次の三つです。

値オブジェクト

業務で扱う値を表現する部品です。同じ値を持つオブジェクトは同じものとして扱います。ですから、IDフィールドは必要ありません。フィールドへの変更は、別の値を生成することを意味します。つまり、値オブジェクトは不変（イミュータブル）なオブジェクトです。

値オブジェクトはデータとロジック（振る舞い）を表現します。つまり、メソッドは値を操作し、その結果として、新たな値オブジェクトを生成します。

集約

トランザクションの境界を共有するエンティティの階層構造です。集約の境界内部のすべてのデータは、集約が表現する業務ロジックを正しく実装するために、強い一貫性を保証する必要があります。

集約の状態および集約内部に保持するオブジェクトの状態を変更する方法は、外部に公開したインターフェースを通じた、コマンドの実行だけです。集約のデータフィールドは外部には読み取りだけを許可します。そうすることで集約に関係するすべての業務ロジックを集約の内部に閉じ込めます。

集約はトランザクションの境界を定義します。集約内部のすべてのデータとオブジェクトの状態は、単一のトランザクションとしてデータベースにコミットします。

集約は外部のエンティティに対して業務イベントを発行して情報を伝達できます。業務イベントとは、集約のライフサイクルで起きた出来事を表現したメッセージです。集約外部のコンポーネントは業務イベントを購読し、業務イベントを受け取ってそれぞれの業務ロジックを実行します。

業務サービス

業務ロジックの置き場所となる、状態を持たないオブジェクトです。集約や値オブジェクトにカプセル化することが不自然な業務ロジックを記述するための部品です。

　業務ロジックの複雑さに立ち向かうために、その複雑さを値オブジェクトと集約の境界内部にカプセル化し、それらを部品としてドメインモデルを組み立てます。オブジェクトの状態を外部から直接変更できないようにすることで、密接に関係するすべての業務ロジックを集約と値オブジェクトの境界内部に一元的に実装し、業務ロジックの記述がアプリケーション層で重複しないようにします。

次の章では、ドメインモデルのより進んだ実装方法を学びます。ドメインモデルの本質的な要素として、時間軸に注目する方法です。

6.4　演習問題

1. どの説明が正しいですか？
   A. 値オブジェクトはデータだけを持つ
   B. 値オブジェクトは振る舞い（ロジック）だけを持つ
   C. 値オブジェクトは不変（イミュータブル）である
   D. 値オブジェクトの内部状態を変えてもよい
2. 集約の境界を設計する一般原則はどれですか？
   A. データベースのトランザクション単位と一つのエンティティを対応させるために、集約が持つエンティティは一つだけにする
   B. 業務データの一貫性を保証できる限り、集約はできるだけ小さく設計する
   C. 集約は複数のエンティティの階層構造である。したがって、データの一貫性を保証するにはできるだけ大きな集約が望ましい
   D. 状況による。つまり、ある業務領域では小さな集約が最善となり、別の業務領域ではできるだけ大きな集約がよい設計になる
3. 集約のインスタンスを単一のトランザクションとしてコミットする理由はどれですか？
   A. 負荷が高い状況での処理性能を保証するため
   B. トランザクションの境界を正しく設定するため
   C. そうする必要はない（対象とする業務領域によって異なる）
   D. 複数レコードのトランザクション管理ができないKey-Value型やドキュメント型のデータベースに対応できるようにするため
4. ドメインモデルを構成する部品について正しい説明はどれですか？
   A. 値オブジェクトはエンティティの属性を記述する
   B. 値オブジェクトは業務イベントを発行できる
   C. 集約は一つまたは複数のエンティティを含む
   D. AとC
5. アクティブレコードと集約の違いの正しい説明はどれですか？
   A. アクティブレコードはデータだけを持つが、集約はデータと振る舞い（ロジック）の両方を持つ
   B. 集約は関連するすべての業務ロジックをカプセル化するが、アクティブレコードでは業務ロジックを別のオブジェクトに記述することが可能
   C. 集約はデータだけを持つが、アクティブレコードはデータと振る舞い（ロジック）の両方を持つ
   D. 集約は内部に複数のアクティブレコードを持つ